

Algorithmes récursifs et interface graphique

TME 7

Objectifs

- ★ Algorithmes récursifs
- ★ Pygame

Rendu

- ⇒ Les réponses seront écrites dans un fichier `nom.n°etu.tp7` où `nom` est votre nom et `n°etu` est votre numéro étudiant.
- ⇒ Si vous avez plusieurs fichiers :
 - Créer un répertoire `rendu TP7_votrenom.n°etu` où `votrenom` est votre nom et `n°etu` est votre numéro étudiant.
 - Copier l'ensemble des fichiers à rendre dans ce répertoire.
 - Archivez le répertoire à l'aide de la commande suivante

```
tar czvf tp7_votrenom.n°etu.tgz TP7_votrenom.n°etu
```
- ⇒ Charger votre fichier sur Moodle dans la devoir TP7

1 Petites exercices récursifs

Exercice 1 Longueur d'une chaîne de caractère

Définir une fonction récursive `lgr(s)` qui détermine la longueur d'une chaîne de caractères `s`.

Exercice 2 Suite de Fibonacci

Proposer une fonction récursive pour calculer la suite de Fibonacci. Pour rappel la suite de Fibonacci est une suite d'entiers dans laquelle chaque terme est la somme des deux termes qui le précèdent. Cette exemple est pour s'entraîner mais pour rappel utiliser la récursivité pour la suite de Fibonacci n'est pas optimal. Sauf si on utilise de techniques un peu plus avancées. La complexité de la solution simple est exponentielle $O(2^n)$. Alors que si la fonction est faite avec une boucle la complexité sera linéaire $O(n)$.

Exercice 3 Fin du Démineur

Modifier la fonction de découverte pour permettre la découverte récursive des cases : quand on découvre une case qui n'est adjacente d'aucune mine, on va découvrir toutes les cases autour, si une d'elle n'a pas non plus de mine adjacente on continue .

2 Interface graphique

Nous allons utiliser la bibliothèque `pygame` pour faire une version graphique du démineur.

Dans une première implémentation il ne sera possible que de jouer un niveau facile avec un plateau de 9x9 et 10 mines. Vous pourrez par la suite ajouter d'autres niveaux et ajouter les éléments graphiques nécessaires pour le choix du niveau de difficulté.

Exercice 4 Initialisation

1. Créez un nouveau répertoire demineur, copiez y votre fichier contenant le code du démineur.
2. Dans un nouveau fichier, importez votre démineur.
3. Importez, ensuite, la bibliothèque `pygame` et initialisez la avec la méthode suivante :

```
pygame.init()
```

4. Nous allons ensuite définir les constantes utiles à l'interface graphique les nombres sont en pixel :

```
SCREEN_WIDTH = 920    # window width
SCREEN_HEIGHT = 920   # window height
MARGIN = 10          # margin around the board
BOARD = 900          # Real size of the board
```

5. Vous pouvez aussi définir les couleurs de votre jeu. Les couleurs sont représentées par des tuples de 3 valeurs RGB. La première valeur détermine le niveau de rouge, la seconde le niveau de vert et B le niveau de bleu. Chaque nombre est compris entre 0 et 255. Vous pouvez choisir vos couleurs ici.

```
LIGHT_GRAY = (224, 224, 224)
BLACK = (0, 0, 0)
WHITE = (255, 255, 255)
...
```

6. On initialise ensuite notre jeu, avec le bon nombre de mine et la taille du plateau de jeu. Vous pouvez utiliser aussi des constantes qui pourront être facilement modifiées si vous voulez augmenter le niveau de difficulté de votre jeu.

Exercice 5 Affichage d'une fenêtre

1. Pour afficher une fenêtre il faut d'abord donner la taille de la fenêtre en utilisant la fonction `pygame` :

```
# Set up the drawing window
screen = pygame.display.set_mode([SCREEN_WIDTH, SCREEN_HEIGHT])
```

Cette fonction nous retourne une *surface* sur lequel on va pouvoir dessiner.

2. Nous allons pour l'instant afficher la fenêtre seule avec le code suivant :

```
running = True
while running:
    # Did the user click the window close button?
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            running = False
    #Refresh the screen
    screen.fill(BLACK)
    pygame.display.flip()

    # deactivates the pygame library
pygame.quit()

# quit the program
quit()
```

Le programme va donc afficher notre écran avec une boucle infinie jusqu'à que l'évènement *quit* arrive. Si c'est le cas on ferme simplement la fenêtre.

Pour mettre à jour l'affichage, nous avons toujours besoin d'effacer tout et de réafficher, comme cela va très vite l'humain n'a pas le temps de voir que l'image a été effacée. Cela permet de ne pas laisser les traces des anciens affichages.

Exercice 6 Affichage des cases simple

Écrire une fonction qui dessine les cases du plateau de jeu. Cette fonction prendra en paramètre la fenêtre d'affichage (*screen*), le tableau des mines et le tableau de statut.

On commence par initialiser la taille d'une case (`BOARD//len(board)`) où *board* est votre plateau de jeu. Ensuite pour chaque case on va créer un `rect`

```
pygame.draw.rect(screen, YOUR_COLOR, [x, y, tile_size-2, tile_size-2])
```

En interface graphique les coordonnées (0,0) se trouvent en haut à gauche. Le rectangle est placé selon son coin gauche. le -2 pixel permet de faire le quadrillage.

Exercice 7 Affichage selon le statut

Modifier votre fonction précédente pour que l’affichage prenne en compte le statut des cases :

- COVER : affichage du rectangle
- UNCOVERED : affichage du chiffre si ce n’est pas une mine :

```
# create a new Font object from a file
font = pygame.font.SysFont(name, size)
# draw text on the new surface
number = font.render(str, True, COLOR)
# draw the new surface on our screen
screen.blit(number, position x, position y)
```

Vous pouvez trouver la liste des fonts disponibles dans votre système par :

```
>>> import pygame
pygame 1.9.4.post1
Hello from the pygame community. https://www.pygame.org/contribute.html
>>> pygame.font.get_fonts()
['lato', 'comicsansms', 'accanthisadfst', 'latinmodernmonolight',
'dejavuserif', ...]
```

- MARK : affichage d’une image que vous aurez au préalable dessiner ou récupérer.

```
# load image can be done as global var (outside the function)
flag_img = pygame.image.load('flag.png')
# resize image can be done as global var (outside the function)
flag_img = pygame.transform.scale(flag_img, (tile_size-2, tile_size-2))
# inside the function
screen.blit(flag_img, (x, y))
```

Exercice 8 Interactions avec le joueur

Pour avoir un jeu complet, il faut ajouter à la boucle d’affichage la gestion des événements :

- cliquez sur une case découvre la case !
- cliquez sur une bombe : on a perdu !
- cliquez droit sur une case couverte la marque avec un drapeau.

Pour cela, selon le clic il suffit d’utiliser vos fonctions de gestion de jeu. La fonction d’affichage fera le reste (sauf dans le cas où on a perdu).

L’évènement clic gauche est reconnu par `event.button == 1` et clic droit par `event.button == 3`.

Pour finir, vous pouvez ajouter le cas où le joueur a gagné, vous pouvez aussi ajouter une image voulez vous continuer ou choix du niveau ou de jolies images. Amusez-vous bien !